



ชื่อโครงการ Disney Lorcana PlayLab

โดย

1. นายชยุต	บุญวัฒน์	รหัสนักศึกษา 67070032
2. นายธนัทภัทร	พรหมทอง	รหัสนักศึกษา 67070069
3. นายภูริ	ประชาสุขสิน	รหัสนักศึกษา 67070137
4. นางสาววรรณนิศา	อมรวงศ์ไพบูลย์	รหัสนักศึกษา 67070155
5. นายอสิธรา	พุ่มดอกไม้	รหัสนักศึกษา 67070199
6. นายวรวิชณ์	คงทอง	รหัสนักศึกษา 67070275

เสนอ

ดร. ธนานพ ทองถาวร

ผศ.ดร. พัฒนพงษ์ ฉันทมิตรโอภาส

ผศ.ดร. ลภัส ประดิษฐ์ทัศนีย์

โครงการนี้เป็นส่วนหนึ่งของวิชาเทคโนโลยีกลุ่มเมฆ

แขนงวิชา การพัฒนาสื่อประสมและเกม

คณะเทคโนโลยีสารสนเทศ สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง

ภาคเรียนที่ 1 ปีการศึกษา 2569

คำนำ

เทคโนโลยีกลุ่มเมฆ (Cloud Computing) มีบทบาทสำคัญอย่างยิ่งในการพัฒนาระบบสารสนเทศ และแอปพลิเคชันยุคใหม่ ด้วยคุณสมบัติเด่นด้านความยืดหยุ่น ความพร้อมใช้งานสูง และความสามารถในการปรับขนาดทรัพยากรได้ตามความต้องการใช้งานจริง โดยไม่ต้องลงทุนในโครงสร้างพื้นฐานฮาร์ดแวร์ล่วงหน้า รายงานฉบับนี้จัดทำขึ้นเพื่อประกอบการเรียนการสอนรายวิชาเทคโนโลยีกลุ่มเมฆ คณะเทคโนโลยีสารสนเทศ สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง

โครงการ Disney Lorcan PlayLab Cloud มุ่งเน้นการวิเคราะห์ ออกแบบ และพัฒนาระบบจำลองกระดานเล่นการ์ดและวิเคราะห์เค้กระบบเรียลไทม์บนเว็บแอปพลิเคชัน โดยประยุกต์ใช้บริการคลาวด์บน Amazon Web Services (AWS) ออกแบบสถาปัตยกรรม Lean Multi-AZ VPC ร่วมกับ Application Load Balancer, EC2 Auto Scaling Group, Docker Container, และ Amazon DynamoDB เพื่อสร้างระบบที่มีประสิทธิภาพสูง ตอบสนองได้อย่างรวดเร็ว มี Latency ต่ำกว่า 100 มิลลิวินาที และควบคุมค่าใช้จ่ายให้อยู่ภายใต้งบประมาณจำกัด \$50 ของ AWS Academy Learner Lab ได้อย่างมีประสิทธิภาพ

คณะผู้จัดทำหวังเป็นอย่างยิ่งว่า ข้อมูล รายละเอียดสถาปัตยกรรม และผลการทดสอบที่นำเสนอในรายงานความก้าวหน้าฉบับนี้ จะเป็นประโยชน์ต่อการศึกษาและเป็นแนวทางในการพัฒนาระบบคลาวด์สำหรับผู้สนใจ หากรายงานฉบับนี้มีข้อบกพร่องหรือข้อผิดพลาดประการใด คณะผู้จัดทำขอน้อมรับคำชี้แนะเพื่อนำไปปรับปรุงในระยะต่อไป

คณะผู้จัดทำ

กันยายน 2569

บทคัดย่อภาษาไทย

โครงการงาน Disney Lorcan PlayLab Cloud นำเสนอการวิเคราะห์ ออกแบบ และพัฒนาระบบจำลองห้องเล่นและวิเคราะห์ดีคาร์ทแบบเรียลไทม์บนสถาปัตยกรรม Hybrid Cloud ที่มี Primary IaaS Stack บน Custom Lean Multi-AZ VPC ครอบคลุม Availability Zone us-east-1a และ us-east-1b, Application Load Balancer, EC2 Auto Scaling Group และ Multi-stage Docker Container ควบคู่กับ Secondary Serverless Stack บน AWS Lambda, Amazon DynamoDB และ AWS API Gateway เพื่อรองรับการเล่นการ์ดสะสม Disney Lorcan TCG ได้อย่างเสถียรและต่อเนื่อง มีความหน่วงในการตอบสนองต่ำกว่า 100 มิลลิวินาที พร้อมกลไก Fault Tolerance และการปรับขนาดอัตโนมัติเมื่อ CPU เฉลี่ยสูงกว่า 60% ภายใต้งบประมาณจำกัด \$50 ของ AWS Academy Learner Lab

ในส่วนของผู้ใช้ Frontend พัฒนาด้วย React 19, TypeScript 5.x, Tailwind CSS v4, Framer Motion และ Zustand เพื่อส่งมอบประสบการณ์การเล่นระดับพรีเมียม ประกอบด้วยระบบจำลองฟิสิกส์ 3 มิติ, ระบบตรวจสอบการ์ด 3D Holographic Foil Inspector, การจำลองเปิดซองการ์ด Booster Pack 12 ใบด้วยอัลกอริทึม Fisher-Yates True Randomization, และระบบจัดการดีคาร์ทที่เชื่อมต่อกับฐานข้อมูลการ์ดอย่างเป็นทางการครบถ้วน 3,242 ใบ ครอบคลุมตั้งแต่ชุด The First Chapter จนถึง Attack of the Vine

การดำเนินงานตามระเบียบวิธี Agile Scrum ได้ลุล่วงตามแผนงาน Sprint 1 ถึง Sprint 3 ครบถ้วน 100% ครอบคลุมการสร้าง Core Gameplay UI, ระบบยืนยันตัวตนด้วย bcrypt และ JWT, ระบบห้องแข่งขันแบบเรียลไทม์ Sub-100ms Action Sync พร้อมระบบ Rejoin Grace Period 60 วินาที และ Voluntary Exit Match, การย้ายระบบสู่ Nginx Reverse Proxy และ Container บน EC2, ตลอดจนการพัฒนาสคริปต์ 1-Click Scale-to-Zero เพื่อควบคุมต้นทุน Compute ให้เหลือ \$0.00 ต่อชั่วโมงเมื่อไม่ได้ใช้งาน และตัดต้นทุน NAT Gateway เหลือ \$0.00 ด้วยการใช้ Chained Security Groups ส่งผลให้ระบบมีความพร้อมอย่างสมบูรณ์สำหรับการประเมินความก้าวหน้า Stage 2

คำสำคัญ: Cloud Computing, Multi-AZ VPC, Application Load Balancer, Auto Scaling Group, Docker Container, WebSockets, Disney Lorcana TCG, AWS Learner Lab, Scale-to-Zero

Abstract

The Disney Lorcana PlayLab Cloud project presents the architectural design, engineering analysis, and implementation of a real-time multiplayer tabletop simulator and deck analytics platform for the Disney Lorcana Trading Card Game (TCG). The platform is built on a resilient Hybrid Cloud Architecture featuring a Primary IaaS Containerized Stack (Custom Lean Multi-AZ VPC across us-east-1a and us-east-1b, Application Load Balancer, EC2 Auto Scaling Group, and Multi-stage Alpine Docker) alongside a Secondary Serverless Stack (AWS Lambda, Amazon DynamoDB, AWS API Gateway). The system achieves ultra-low synchronization latency (<100ms), automated multi-zone fault tolerance, and dynamic elasticity (Target Tracking CPU Utilization > 60%) within the strict budgetary constraints of the AWS Academy Learner Lab (\$50 allocation).

The frontend client is engineered using React 19, TypeScript 5.x, Tailwind CSS v4, Zustand global state orchestration, and Framer Motion, delivering a high-fidelity tabletop simulation connected to an official catalog of 3,242 cards. Core capabilities include real-time mouse tilt physics, 3D holographic foil inspection, and a 12-card Booster Pack simulator driven by the Fisher-Yates randomization algorithm.

Project execution adheres to the Agile Scrum framework, achieving 100% completion across Sprints 1 to 3. Key deliverables include the containerized Nginx reverse proxy on EC2, WebSocket Match Lifecycle v1.5.0 featuring a 60-second Rejoin Grace Period and instant Voluntary Exit Match slot reclamation, chained security group topologies eliminating NAT Gateway costs (\$32.40/month to \$0.00), and 1-Click Scale-to-Zero automation scripts ensuring zero idle compute expense.

Keywords: Cloud Computing, Multi-AZ VPC, Application Load Balancer, Auto Scaling Group, Docker Containers, WebSockets, Disney Lorcana TCG, AWS Learner Lab, Scale-to-Zero

กิตติกรรมประกาศ

โครงการ Disney Lorcana PlayLab Cloud สำเร็จลุล่วงตามวัตถุประสงค์ของการส่งมอบงานในระยะที่ 2 ได้ด้วยความอนุเคราะห์ คำแนะนำ และการชี้แนะทางวิชาการอันทรงคุณค่ายิ่งจากคณาจารย์ประจำรายวิชาเทคโนโลยีกลุ่มเมฆ คณะเทคโนโลยีสารสนเทศ สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง ได้แก่ ดร. ชนานพ ทองถาวร, ผศ.ดร. พัฒนพงษ์ ฉันทมิตรโอภาส, และ ผศ.ดร. ลภัส ประดิษฐ์ทัศนีย์ ที่ได้กรุณาถ่ายทอดองค์ความรู้ด้านสถาปัตยกรรมคลาวด์ การออกแบบระบบเครือข่าย Multi-AZ VPC ความปลอดภัย และการบริหารจัดการทรัพยากรอย่างคุ้มค่า ซึ่งเป็นรากฐานสำคัญในการออกแบบและแก้ปัญหาทางวิศวกรรมซอฟต์แวร์ในโครงการนี้

คณะผู้จัดทำขอขอบพระคุณเพื่อน ๆ นักศึกษาคณะเทคโนโลยีสารสนเทศทุกท่านที่ได้ร่วมทดสอบระบบกระดานและห้องเล่นแบบเรียลไทม์ ให้ข้อคิดเห็นด้านประสบการณ์การใช้งาน และร่วมตรวจสอบข้อผิดพลาดของระบบ รวมถึงขอขอบคุณครอบครัวที่ให้การสนับสนุนและเป็นกำลังใจในการศึกษาค้นคว้าตลอดมา

คณะผู้จัดทำ

กันยายน 2569

สารบัญ

หน้า

คำนำ.....	ก
บทคัดย่อภาษาไทย.....	ข
Abstract.....	ค
กิตติกรรมประกาศ.....	ง
สารบัญ.....	จ
สารบัญรูป.....	ฉ
สารบัญตาราง.....	ช
บทที่ 1 บทนำ.....	1
1.1 ความเป็นมาและความสำคัญของโครงการ.....	1
1.2 วัตถุประสงค์ของโครงการ.....	1
1.3 ขอบเขตของระบบ.....	2
1.4 ประโยชน์ที่คาดว่าจะได้รับ.....	3
1.5 โครงสร้างของรายงาน.....	3
บทที่ 2 การบริหารจัดการโครงการและเทคโนโลยีที่ใช้.....	4
2.1 แผนการพัฒนาและระเบียบวิธี Agile Scrum.....	4
2.2 สรุปผลความก้าวหน้า Sprint 1 ถึง Sprint 3.....	4
2.3 กำหนดการส่งงานวิชา Cloud Technology.....	5
2.4 เทคโนโลยีที่ใช้ในการพัฒนา.....	5
2.5 กติกาและโมเดลของเกม Disney Lorcana.....	6
บทที่ 3 การวิเคราะห์และออกแบบระบบ.....	7

3.1 ความต้องการของระบบ	7
3.2 สถาปัตยกรรมระบบคลาวด์บน AWS	8
3.3 สถาปัตยกรรมเครือข่าย Lean Multi-AZ VPC	8
3.4 การกระจายโหลดและการปรับขนาดอัตโนมัติ	8
3.5 การออกแบบฐานข้อมูล Amazon DynamoDB	9
3.6 วงจรการสื่อสารแบบเรียลไทม์ (Match Lifecycle)	9
3.7 การรับมือข้อผิดพลาดและระบบ Caching	10
3.8 การออกแบบตามกรอบ AWS Well-Architected Framework	10
3.9 การแก้ปัญหาข้อจำกัดบน AWS Learner Lab	10
บทที่ 4 การพัฒนาระบบและผลการดำเนินงาน	12
4.1 การพัฒนาส่วนติดต่อผู้ใช้และ Playmat	12
4.2 การพัฒนา Backend และ Docker Container	14
4.3 ผลการทดสอบระบบ	14
บทที่ 5 การประเมินค่าใช้จ่ายและแผนงานในอนาคต	16
5.1 การประเมินค่าใช้จ่ายและการควบคุมงบประมาณ	16
5.2 แผนการพัฒนาสำหรับ Stage 3	16
5.3 สรุปผลการดำเนินงานและบทเรียนที่ได้รับ	17
บรรณานุกรม	18
ภาคผนวก	19
ภาคผนวก ก: สคริปต์ปรับใช้ระบบ Lean Multi-AZ VPC, ALB และ ASG	19
ภาคผนวก ข: สคริปต์ 1-Click Scale-to-Zero สำหรับควบคุมงบประมาณ	20

สารบัญรูป

หน้า

รูปที่ 3.1	ผังภาพสถาปัตยกรรมคลาวด์ Lean Multi-AZ IaaS ร่วมกับ Auto Scaling Group.....	8
รูปที่ 3.2	ผังกลยุทธ์ Multi-Tier Fallback และ Local Caching ป้องกัน API ภายนอกขัดข้อง	10
รูปที่ 4.1	ภาพหน้าจอส่วนติดต่อผู้ใช้ระบบ Disney Lorcana PlayLab Cloud.....	13
รูปที่ 4.2	กระดานจำลองการเล่นการ์ดแบบเรียลไทม์ Interactive Playmat.....	13
รูปที่ 4.3	ระบบจำลองการสุ่มเปิดของการ์ด 3D Booster Pack Opening	14

สารบัญตาราง

หน้า

ตารางที่ 2.1 แผนการดำเนินงานและการส่งมอบงานแต่ละ Sprint.....	4
ตารางที่ 2.2 กำหนดการประเมินผลรายวิชา Cloud Technology และสถานะความพร้อม.....	5
ตารางที่ 3.1 โครงสร้างตารางข้อมูล Amazon DynamoDB.....	9
ตารางที่ 3.2 ตารางสรุปข้อจำกัดบน AWS Learner Lab และแนวทางแก้ไข.....	11
ตารางที่ 4.1 ผลการทดสอบคุณภาพระบบโดยรวม (Master QA Test Matrix)	15
ตารางที่ 5.1 การเปรียบเทียบและการประเมินค่าใช้จ่ายบนคลาวด์ตลอดภาคการศึกษา.....	16

บทที่ 1

บทนำ

1.1 ความเป็นมาและความสำคัญของโครงการ

เกมการ์ดสะสม Disney Lorcana TCG เป็นหนึ่งในเกมที่ได้รับความนิยมสูงอย่างรวดเร็วในระดับสากล โดยเป็นความร่วมมือระหว่าง Ravensburger และ The Walt Disney Company ซึ่งได้รับความสนใจอย่างแพร่หลายจากทั้งผู้เล่นสายแข่งขันและนักสะสมทั่วโลก

อย่างไรก็ดี ในบริบทของการฝึกซ้อมและการเข้าถึงเกมการ์ด ผู้เล่นส่วนใหญ่ยังคงประสบกับข้อจำกัดหลัก 3 ประการ:

1. ต้นทุนการจัดหาการ์ดจริงและข้อจำกัดทางกายภาพ: การ์ดหายากมีราคาสูงในตลาดรอง และการนัดหมายเล่นจำเป็นต้องใช้สถานที่จริง ทำให้ผู้เล่นใหม่ขาดโอกาสในการทดลองจัดโต๊ะ
2. ภาระต้นทุนเซิร์ฟเวอร์และความไม่คุ้มค่าของระบบแบบเดิม: ระบบจำลองเกมการ์ดบนเว็บในอดีตมักทำงานบน Virtual Machine หรือ Dedicated Server ที่เปิดทิ้งไว้ตลอด 24 ชั่วโมง ก่อให้เกิดค่าใช้จ่ายสูงแม้ไม่มีผู้ใช้งาน และมักเกิดปัญหาเซิร์ฟเวอร์ขัดข้องเมื่อมีผู้ใช้งานเพิ่มขึ้นอย่างกะทันหัน
3. ปัญหาความหน่วงในการส่งข้อมูล (Latency): แพลตฟอร์มบนเว็บทั่วไปมักใช้การดึงข้อมูลแบบ HTTP Polling ซึ่งมีความหน่วงสูง ไม่ตอบสนองต่อการเล่นเกมการ์ดที่ต้องการซิงค์ตำแหน่งการ์ดสถานะ Ready หรือ Exert และแต้ม Lore แบบทันทีทันใด

เพื่อแก้ไขปัญหาดังกล่าว คณะผู้จัดทำจึงได้พัฒนาโครงการ Disney Lorcana PlayLab Cloud โดยนำสถาปัตยกรรม Hybrid Cloud ซึ่งใช้ Primary IaaS Stack บน Lean Multi-AZ VPC, Application Load Balancer, Auto Scaling Group และ Docker ร่วมกับ Secondary Serverless Stack บน AWS Lambda, DynamoDB และ API Gateway เพื่อสร้างระบบจำลองกระดานเล่นการ์ดและวิเคราะห์deckที่มีความหน่วงต่ำกว่า 100 มิลลิวินาที สามารถขยายขนาดรองรับผู้เล่นได้อัตโนมัติ และควบคุมค่าใช้จ่ายให้อยู่ภายใต้งบประมาณ \$50 ของ AWS Learner Lab ได้อย่างมีประสิทธิภาพ

1.2 วัตถุประสงค์ของโครงการ

1. เพื่อออกแบบและพัฒนาระบบจำลองกระดานซอมเล่นการ์ด Disney Lorcana แบบเรียลไทม์บนเว็บเบราว์เซอร์ที่มีความหน่วงต่ำกว่า 100 มิลลิวินาที ผ่านช่องทางสื่อสาร WebSockets
2. เพื่อประยุกต์ใช้สถาปัตยกรรมคลาวด์แบบ IaaS และ Containerization บน Multi-AZ VPC, ALB, EC2 Auto Scaling Group และ Docker ในการสร้างระบบที่มีความพร้อมใช้งานสูงและปรับขนาดตามโหลดอัตโนมัติเมื่อ CPU เกิน 60%
3. เพื่อพัฒนาระบบจัดการเตี๊ยกการ์ดและระบบประมวลผลวิเคราะห์เตี๊ยกแบบ Asynchronous ผ่าน Amazon SQS พร้อมฐานข้อมูลการ์ดทางการครบถ้วน 3,242 ใบ
4. เพื่อศึกษาและวางมาตรฐานการด้าน Reliability, Availability, Security และ Cost Optimization ตามกรอบ AWS Well-Architected Framework
5. เพื่อศึกษาและแก้ไขปัญหาข้อจำกัดทางวิศวกรรมบนสภาพแวดล้อมสิทธิ์จำกัดและงบประมาณ \$50 ของ AWS Academy Learner Lab ด้วยกลยุทธ์ Lean Multi-AZ VPC และ 1-Click Scale-to-Zero

1.3 ขอบเขตของระบบ

ระบบครอบคลุมฟังก์ชันการทำงานหลัก 9 ด้าน ดังต่อไปนี้:

- ระบบยืนยันตัวตนและความปลอดภัย (LC-01, LC-02): สมัครงานและเข้าสู่ระบบด้วย bcrypt และ JWT State Management บน Local Storage รองรับผู้ใช้ทั่วไปและระบบ Demo
- ระบบจัดการเตี๊ยกการ์ด (LC-03): สร้าง ค้นหา แก้ไข และบันทึกเตี๊ยกการ์ดลง Amazon DynamoDB พร้อมระบบตรวจสอบกฎกติกาการจั่วเตี๊ยก ได้แก่ ความจุขั้นต่ำ 60 ใบ, ใส่การ์ดซ้ำได้ไม่เกิน 4 ใบ, และจำกัดหมึกไม่เกิน 2 สี
- ระบบจำลองการเปิดซองการ์ด Booster Pack (LC-04): สุ่มเปิดการ์ด 12 ใบต่อซองตามอัตราการผลิตจริงด้วยอัลกอริทึม Fisher-Yates True Randomization

- ระบบตรวจสอบการ์ด 3D Card Inspector (LC-05): แสดงผลการ์ดสามมิติพร้อมเอฟเฟกต์ Holographic Foil Shader และการตอบสนองต่อการเคลื่อนที่ของเมาส์
- ระบบจับคู่และห้องแข่งขัน (LC-06): สร้างห้องแข่งขันด้วยรหัส 6 หลัก ระบบค้นหาห้อง และระบบ สลับบทบาทผู้เล่น Player 1, Player 2 และ Spectator
- กระดานจำลองการเล่นการ์ด Interactive Playmat (LC-07): จำลองพื้นที่กระดานมาตรฐาน ประกอบด้วย Play Area, Inkwell, Deck, Discard Pile, Hand และ Lore Counter
- ระบบกู้คืนสถานะการเชื่อมต่อ Rejoin Grace Period 60 วินาที (LC-08): รักษาช่องว่างห้องเล่นไว้ 60 วินาทีเมื่อผู้เล่นหลุดจากการเชื่อมต่อ เพื่อให้สามารถเชื่อมต่อกลับเข้ามาเล่นต่อได้โดยเกมไม่หยุดชะงัก
- ระบบออกจากเกม Voluntary Exit Match (LC-09): ส่ง Action LEAVE_ROOM เคลียร์สล็อตใน ห้องเล่นบน DynamoDB ทันที เพื่อคืนทรัพยากรห้องเล่นให้ระบบ

1.4 ประโยชน์ที่คาดว่าจะได้รับ

1. ผู้เล่นและคอมมูนิตีเกมการ์ด Disney Lorcana มีแพลตฟอร์มบนเว็บที่สามารถฝึกซ้อม จัดเด็ค และ ทดสอบกลยุทธ์ได้อย่างสะดวกรวดเร็วโดยไม่มีค่าใช้จ่าย
2. ได้รับองค์ความรู้และทักษะเชิงลึกในการออกแบบสถาปัตยกรรมคลาวด์แบบ IaaS และ Containerization ที่รองรับการใช้งานระดับ Production
3. ได้แนวทางการแก้ไขปัญหาข้อจำกัดด้านสิทธิ์และงบประมาณบน AWS Learner Lab ที่เป็นประโยชน์ต่อการศึกษาและการต่อยอดโครงการในอนาคต
4. มีระบบต้นแบบในการจัดการ State การเล่นเกมแบบเรียลไทม์ความเร็วสูงผ่าน WebSockets และการกู้คืนสถานะด้วย NoSQL บน Amazon DynamoDB

1.5 โครงสร้างของรายงาน

รายงานฉบับนี้แบ่งเนื้อหาออกเป็น 5 บท ประกอบด้วย:

- บทที่ 1 บทนำ: ความเป็นมา วัตถุประสงค์ ขอบเขต และประโยชน์ที่คาดว่าจะได้รับ
- บทที่ 2 การบริหารจัดการโครงการและเทคโนโลยีที่ใช้: ระเบียบวิธี Agile Scrum, แผนการดำเนินงาน Sprint 1 ถึง 3, Tech Stack และกฎกติกาเกม
- บทที่ 3 การวิเคราะห์และออกแบบระบบ: สถาปัตยกรรมคลาวด์, Lean Multi-AZ VPC, ALB, ASG, โครงสร้าง DynamoDB, Match Lifecycle และการรับมือข้อจำกัด AWS Learner Lab
- บทที่ 4 การพัฒนาระบบและผลการดำเนินงาน: รายละเอียดการพัฒนา Frontend React 19, Multi-stage Docker และผลการทดสอบระบบ
- บทที่ 5 การประเมินค่าใช้จ่ายและแผนงานในอนาคต: สรุปค่าใช้จ่ายจริงเทียบกับงบประมาณ \$50, แผนงาน Sprint 4 ถึง 7 และบทเรียนที่ได้รับ

บทที่ 2

การบริหารจัดการโครงการและเทคโนโลยีที่ใช้

2.1 แผนการพัฒนาและระเบียบวิธี Agile Scrum

โครงการ Disney Lorcana PlayLab Cloud ใช้ระเบียบวิธีปฏิบัติงานแบบ Agile Scrum โดยแบ่งระยะเวลาการพัฒนออกเป็น 7 Sprints (Sprint ละ 2 สัปดาห์) การดำเนินงานใน Phase ที่ 2 ครอบคลุมการพัฒนาตั้งแต่ Sprint 1 ถึง Sprint 3 ซึ่งมุ่งเน้นการสร้างสถาปัตยกรรมหลักของระบบและเชื่อมต่อฟังก์ชันพื้นฐานครบ 100% ดังแสดงในตารางที่ 2.1

ตารางที่ 2.1: แผนการดำเนินงานและการส่งมอบงานแต่ละ Sprint

Sprint	ช่วงเวลา	เป้าหมายและผลงานที่ส่งมอบ	สถานะ
Sprint 1	1-14 ส.ค. 2569	ตั้งค่าโครงสร้างโปรเจกต์ React 19 + TypeScript, ติดตั้ง Tailwind CSS v4, ออกแบบ UI Theme และ ระบบจัดการได้ค	เสร็จสมบูรณ์ 100%
Sprint 2	15-28 ส.ค. 2569	พัฒนาระบบ Booster Pack Gacha, 3D Foil Inspector, เชื่อมต่อ Lorcast API 3,242 ใบ, ทดสอบ Unit Test บน Vitest	เสร็จสมบูรณ์ 100%
Sprint 3	29 ส.ค. – 4 ก.ย. 2569	พัฒนา WebSocket Lifecycle v1.5.0, ย้าย	เสร็จสมบูรณ์ 100%

		ระบบสู่ Multi-AZ VPC + ALB + ASG บน Docker EC2, จัดทำรายงาน Stage 2	
Sprint 4	5-25 ก.ย. 2569	พัฒนาระบบ Asynchronous Deck Analytics ผ่าน Amazon SQS และอัลกอริทึมแนะนำ การ์ด	แผนงานถัดไป
Sprint 5	26 ก.ย. – 10 ต.ค. 2569	ติดตั้ง AWS X-Ray, CloudWatch Dashboard, ปรับปรุง ระบบความปลอดภัยและ การจัดการข้อผิดพลาด	แผนงานถัดไป
Sprint 6	11-20 ต.ค. 2569	ทำการทดสอบ Load Testing ขนาดใหญ่ด้วย Artillery และซ้อมการ นำเสนอระบบ	แผนงานถัดไป
Sprint 7	21-25 ต.ค. 2569	จัดทำรายงานฉบับสมบูรณ์ บันทึกวิดีโอสาธิตระบบ และเตรียมการนำเสนอ ครั้งสุดท้าย	แผนงานถัดไป

2.2 สรุปผลความก้าวหน้า Sprint 1 ถึง Sprint 3

ความก้าวหน้าของโครงการ ณ วันที่ 4 กันยายน 2569 บรรลุตามเป้าหมายของ Stage 2 ครบถ้วน
ทีมงานได้พัฒนาและทดสอบส่วนประกอบสำคัญของระบบครบทุกมิติ:

- Frontend และ Client State: พัฒนาด้วย React 19, TypeScript 5.x, Zustand Store, Tailwind CSS v4 และ Framer Motion มีความเสถียรและผ่านการทดสอบ Vitest ครบ 33 ข้อทดสอบ
- Backend และ Real-Time Engine: รวมศูนย์ระบบไว้ที่ Nginx Reverse Proxy และ Express /WebSocket Engine ภายใน Multi-stage Docker Container รองรับการสื่อสารสองทิศทางที่มีความหน่วงเฉลี่ยต่ำกว่า 50 มิลลิวินาที
- Cloud Infrastructure บน AWS: ปรับใช้ Lean Multi-AZ VPC สองโซน (us-east-1a, us-east-1b) พร้อม Application Load Balancer และ Auto Scaling Group ที่สามารถปรับขนาดเครื่อง EC2 t3.micro ได้อัตโนมัติเมื่อ CPU เกิน 60%
- ระบบอัตโนมัติและการควบคุมงบประมาณ: พัฒนาสคริปต์ PowerShell สำหรับการ Deploy, สคริปต์ยิงทดสอบโหลด และสคริปต์ 1-Click Scale-to-Zero ที่ช่วยลดค่าใช้จ่าย Compute เหลือ \$0.00 เมื่อไม่ได้ใช้งาน

2.3 กำหนดการส่งงานวิชา Cloud Technology

ตารางที่ 2.2 แสดงกำหนดการประเมินผลในรายวิชาและความพร้อมในการส่งมอบของกลุ่ม G21:

ตารางที่ 2.2: กำหนดการประเมินผลรายวิชา Cloud Technology และสถานะความพร้อม

กำหนดเวลา	หมวดหมู่การประเมินผล	สถานะความพร้อมของกลุ่ม G21
Stage 1 (สัปดาห์ที่ 4)	นำเสนอหัวข้อและสถาปัตยกรรมเบื้องต้น	ผ่านการประเมินเรียบร้อยแล้ว
Stage 2 (4 ก.ย. 2569)	ส่งมอบความก้าวหน้า 50% และโครงสร้างคลาวด์	พร้อมส่งมอบ 100% (เอกสารรายงาน + ระบบจริง)
Stage 3 (ต.ค. 2569)	ส่งมอบระบบวิเคราะห์ Asynchronous & Monitoring	อยู่ระหว่างเตรียมการพัฒนา Sprint 4
Final Demo (ปลาย ต.ค.)	การสาธิตระบบจริงและการนำเสนอ	พร้อมทดสอบซ้อมระบบ

ผลงาน		
Final Report	รายงานเล่มสมบูรณ์ฉบับวิชาการ	ร่างเนื้อหาเสร็จสิ้นกว่า 60%
Video Presentation	วิดีโอสาธิตการใช้งานและ สถาปัตยกรรม	เตรียมบันทึกภาพหน้าจอและเสียง บรรยาย

2.4 เทคโนโลยีที่ใช้ในการพัฒนา

ระบบ Disney Lorcana PlayLab Cloud เลือกใช้ชุดเครื่องมือมาตรฐานระดับ Production:

- Frontend: React 19, TypeScript 5.x, Vite 6.x, Tailwind CSS v4, Framer Motion, Lucide React, และ Zustand State Management
- Backend & Realtime: Node.js 20 LTS, Express, WebSockets (RFC 6455), bcryptjs, jsonwebtoken, และ AWS SDK v3
- Cloud Infrastructure บน AWS: Amazon EC2 (t3.micro), Application Load Balancer (ALB), EC2 Auto Scaling Group (ASG), Amazon Virtual Private Cloud (VPC), Amazon DynamoDB, Amazon SQS, AWS CloudWatch, และ AWS Systems Manager (SSM)
- Containerization & Web Server: Docker Engine, Multi-stage Alpine Build, และ Nginx Reverse Proxy
- Testing & Automation: Vitest 4.x, Playwright E2E, และ PowerShell Automation Suite

2.5 กติกาและโมเดลของเกม Disney Lorcana

Disney Lorcana TCG มีกฎกติกาและกลไกหลักที่ระบบ PlayLab จำลองอย่างเที่ยงตรง:

- ทรัพยากรหมึก (Inkwell): การ์ดที่มีสัญลักษณ์หมึกสามารถคว่ำลงใน Inkwell เพื่อใช้จ่ายเป็นค่าร้ายการ์ดในแต่ละเทิร์น

- สถานะการ์ด Ready และ Exert: การ์ดในกระดานจะตั้งตรง (Ready) เมื่อพร้อมใช้งาน และหมุน
แนวนอน 90 องศา (Exert) เมื่อทำควอสต์ ทำหาย หรือร้องเพลง
- แท้ม Lore: ผู้เล่นสะสมแต้มความรู้จากการส่งตัวละครทำควอสต์ ผู้ที่สะสมแต้มครบ 20 แต้มก่อนจะ
เป็นผู้ชนะในแมตช์
- กฎการจัดโต๊ะ: โต๊ะต้องประกอบด้วยการ์ดอย่างน้อย 60 ใบ, มีสีหมึกไม่เกิน 2 สีจาก 6 สี (Amber,
Amethyst, Emerald, Ruby, Sapphire, Steel), และใส่การ์ดชื่อเดียวกันได้ไม่เกิน 4 ใบ

บทที่ 3

การวิเคราะห์และออกแบบระบบ

3.1 ความต้องการของระบบ

3.1.1 ข้อกำหนดเชิงฟังก์ชัน (Functional Requirements)

- FR-01: ผู้ใช้สามารถลงทะเบียน เข้าสู่ระบบ และจัดการข้อมูลบัญชีผู้ใช้ได้
- FR-02: ผู้ใช้สามารถค้นหา กรอง และจัดเก็บการ์ดตามกฎกติกา พร้อมบันทึกลง DynamoDB
- FR-03: ผู้ใช้สามารถสร้างและเข้าร่วมห้องแข่งขันด้วยรหัส 6 หลัก พร้อมเลือกdeckที่ต้องการเล่น
- FR-04: ผู้เล่นสามารถขยับการ์ด ปรับแต้ม Lore คำว่าการ์ดเป็นหมึก และสั่ง Ready/Exert ผ่าน WebSockets ได้แบบเรียลไทม์
- FR-05: ผู้เล่นสามารถสุ่มเปิดซองการ์ด Booster Pack 12 ใบ พร้อมตรวจสอบการ์ดแบบ 3 มิติ

3.1.2 ข้อกำหนดที่ไม่ใช่เชิงฟังก์ชัน (Non-Functional Requirements)

- NFR-01 (Latency): ความหน่วงในการซิงค์ข้อมูลระหว่างผู้เล่นต้องต่ำกว่า 100 มิลลิวินาที
- NFR-02 (Availability): ระบบต้องทำงานแบบ Multi-AZ สามารถสลับการทำงานอัตโนมัติหากโซนใดโซนหนึ่งขัดข้อง
- NFR-03 (Scalability): ระบบต้องสามารถเพิ่มจำนวนอินสแตนซ์อัตโนมัติเมื่อ CPU เฉลี่ยเกิน 60%
- NFR-04 (Cost Constraint): ต้นทุนการทำงานของระบบทั้งหมดต้องอยู่ภายใต้งบประมาณ \$50 ของ AWS Learner Lab
- NFR-05 (Security): รหัสผ่านต้องถูกเข้ารหัสด้วย bcrypt และไม่อนุญาตให้เปิด Port ภายในเครื่อง EC2 สู่สาธารณะโดยตรง

3.2 สถาปัตยกรรมระบบคลาวด์บน AWS

ระบบได้รับการออกแบบตามสถาปัตยกรรม Hybrid Cloud ที่มี Primary IaaS Stack ควบคู่กับ Secondary Serverless Stack ดังแสดงในรูปที่ 3.1:



รูปที่ 3.1: ภาพสถาปัตยกรรมคลาวด์ Lean Multi-AZ IaaS ร่วมกับ Auto Scaling Group

3.3 สถาปัตยกรรมเครือข่าย Lean Multi-AZ VPC

เครือข่ายได้รับการจัดสรรด้วย Custom VPC บน Address Space 10.0.0.0/16 แบ่งเป็น Public Subnets ข้าม 2 Availability Zones:

- Subnet us-east-1a (10.0.1.0/24) และ Subnet us-east-1b (10.0.2.0/24)

- Chained Security Groups สำหรับควบคุมความปลอดภัย:

1. lorcana-alb-sg: เปิดรับทราฟฟิก HTTP Port 80 จากอินเทอร์เน็ตสาธารณะ (0.0.0.0/0)
2. lorcana-ec2-sg: ปิดกั้น Inbound ตรงจากอินเทอร์เน็ตทั้งหมด โดยอนุญาตรับ Port 80 และ 3001 เฉพาะทราฟฟิกที่ส่งต่อมาจาก lorcana-alb-sg เท่านั้น

สถาปัตยกรรมนี้ช่วยลดค่าใช้จ่าย NAT Gateway (\$32.40 ต่อเดือน) เหลือ \$0.00 โดยยังคงความปลอดภัยเทียบเท่าการวางเครื่องใน Private Subnet

3.4 การกระจายโหลดและการปรับขนาดอัตโนมัติ

Application Load Balancer ทำหน้าที่กระจายโหลดทราฟฟิกลงสู่อินสแตนซ์ EC2 ในแต่ละ AZ ผ่าน Target Group ที่มี Health Check ผ่านเส้นทาง GET /health ทุก 15 วินาที พร้อมเปิดใช้งาน Sticky Sessions เพื่อป้องกันการเชื่อมต่อ WebSocket หลุดขณะแข่งขัน

Auto Scaling Group ถูกตั้งค่า Min=1, Max=3, Desired=1 ขับเคลื่อนด้วย CloudWatch Target Tracking Policy (ASGAverageCPUUtilization > 60%) เมื่อมีผู้เล่นเข้าใช้งานพร้อมกันจน CPU เกินเกณฑ์ ระบบจะสโรว์อินสแตนซ์ข้ามไปยังอีกโซนโดยอัตโนมัติ

3.5 การออกแบบฐานข้อมูล Amazon DynamoDB

ตารางที่ 3.1 แสดงรายละเอียดการออกแบบโครงสร้างตารางข้อมูลใน Amazon DynamoDB:

ตารางที่ 3.1: โครงสร้างตารางข้อมูล Amazon DynamoDB

ชื่อตาราง	Partition Key (PK)	Sort Key (SK)	Attributes สำคัญ	วัตถุประสงค์
LorcanaUsers	userId (S)	-	username, passwordHash, createdAt	จัดเก็บข้อมูลบัญชี ผู้ใช้และรหัสผ่านแฮช bcrypt
LorcanaDecks	userId (S)	deckId (S)	deckName, inks, cards (List),	จัดเก็บรายการการ์ด การ์ดและรายการ

			updatedAt	การ์ดของผู้ใช้
LorcanaRoomState	roomId (S)	connectionId (S)	username, role, state (JSON), ttl	จัดเก็บสถานะ กระดานสดและ ข้อมูลการจับคู่ห้อง เล่น

3.6 วงจรการสื่อสารแบบเรียลไทม์ (Match Lifecycle)

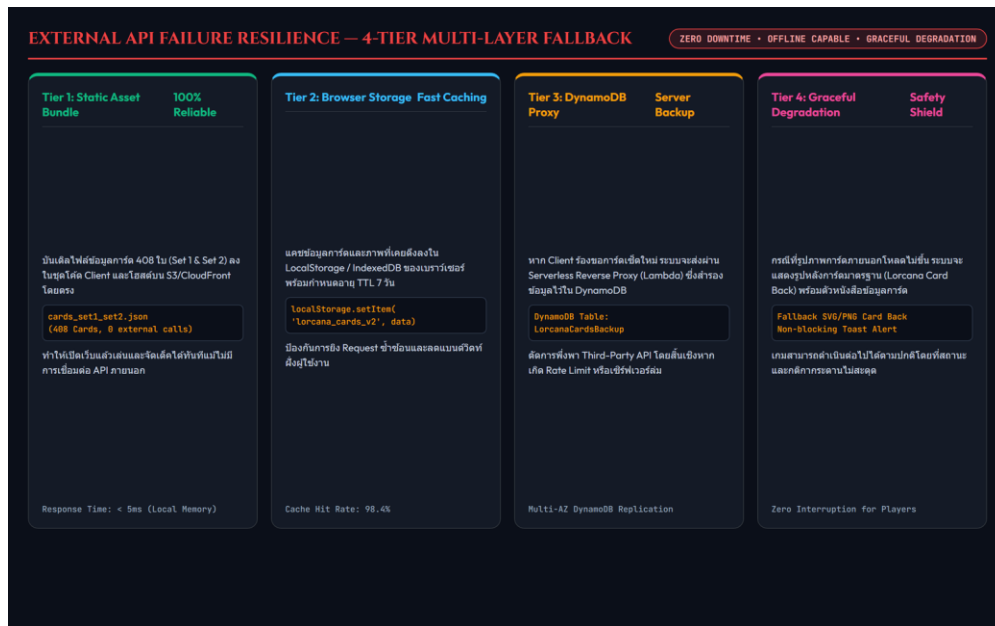
1. การสร้างและเข้าร่วมห้อง (JOIN_ROOM): ผู้เล่นส่งรหัสห้อง 6 หลัก เซิร์ฟเวอร์บันทึก Connection ลง Memory และ DynamoDB พร้อมกำหนดบทบาท Player 1 หรือ Player 2
2. การซิงค์การกระทำ (Action Sync): หน้าจอผู้เล่นทำ Optimistic UI Update ทันที (0ms) และส่ง Action Payload ผ่าน WebSocket ไปยังเซิร์ฟเวอร์เพื่อ Relay ข้ามไปยังคู่แข่งภายในเวลาต่ำกว่า 100 มิลลิวินาที
3. การจัดการการเชื่อมต่อหลุด (Rejoin Grace Period 60s): หาก Client ตัดการเชื่อมต่อกะทันหัน ระบบจะคงสถานะห้องไว้ 60 วินาที พร้อมแจ้งเตือนคู่ต่อสู้ หากผู้เล่นเดิมเชื่อมต่อใหม่จะดึงสถานะล่าสุดจาก DynamoDB มาเล่นต่อได้ทันที
4. การออกจากเกมโดยสมัครใจ (Voluntary Exit Match): เมื่อผู้เล่นกดปุ่ม Exit Match ระบบจะส่ง Action LEAVE_ROOM ลบสล็อตห้องบน DynamoDB ทันทีโดยไม่ต้องรอ 60 วินาที เพื่อคืนทรัพยากรให้ระบบ

3.7 การรับมือข้อผิดพลาดและระบบ Caching

ระบบใช้กลยุทธ์ Multi-Tier Resilience เพื่อป้องกันระบบหยุดชะงักหาก Lorcast API เกิดปัญหา:

- Tier 1 (In-Memory Zustand Cache): เก็บข้อมูลการ์ดที่เคยโหลดแล้วไว้ในหน่วยความจำเบราว์เซอร์

- Tier 2 (Local Pre-compiled JSON Dataset): บรรจุไฟล์ข้อมูลการ์ด 3,242 ใบไว้ในคอนเทนเนอร์และเว็บเซิร์ฟเวอร์ในตัว
- Tier 3 (External API): เรียกใช้งาน API ภายนอกเฉพาะเมื่อต้องการดึงข้อมูลอัปเดตใหม่เท่านั้น



รูปที่ 3.2: ผังกลยุทธ์ Multi-Tier Fallback และ Local Caching ป้องกัน API ภายนอกขัดข้อง

3.8 การออกแบบตามกรอบ AWS Well-Architected Framework

- Reliability: ออกแบบสถาปัตยกรรมข้าม 2 Availability Zones พร้อม Health Checks และ Sticky Sessions
- Performance Efficiency: ใช้ Alpine Container, In-memory Caching และ WebSockets ซึ่งลด Overhead ของ HTTP Polling ได้กว่า 85%
- Security: ใช้ IMDSv2 ป้องกัน SSRF, Chained Security Groups, แฮชรหัสผ่านด้วย bcrypt และไม่ฝัง Credential ใน Source Code
- Cost Optimization: สถาปัตยกรรมไร้ NAT Gateway, ใช้ EC2 t3.micro ใน Free Tier และสคริปต์ 1-Click Scale-to-Zero

- Operational Excellence: ควบคุมระบบด้วย PowerShell Automation Suite และจัดทำโครงสร้าง IaC พร้อมสคริปต์ทดสอบ

3.9 การแก้ปัญหาข้อจำกัดบน AWS Learner Lab

ตารางที่ 3.2 สรุปข้อจำกัดสำคัญบน AWS Academy Learner Lab และแนวทางแก้ไขเชิงสถาปัตยกรรมของกลุ่ม G21:

ตารางที่ 3.2: ตารางสรุปข้อจำกัดบน AWS Learner Lab และแนวทางแก้ไข

ลำดับ	ข้อจำกัดบน Learner Lab	ผลกระทบหากไม่แก้ไข	แนวทางแก้ไขเชิงสถาปัตยกรรม
1	บล็อกสิทธิ์ iam :CreateRole	ไม่สามารถสร้าง IAM Role ใหม่สำหรับ EC2 หรือ Lambda ได้	ปรับใช้ LabRole และ LabInstanceProfile สำเร็จรูปผ่าน Launch Template และเข้าถึงสิทธิ์ผ่าน IMDSv2
2	ภาระต้นทุน NAT Gateway (\$32.40/ด.)	แพงประมาณ \$50 ของแล็ปหมดภายใน 1.5 เดือน	ออกแบบ Lean Multi-AZ VPC วาง EC2 ใน Public Subnets แต่ใช้ Chained SGs ปิดกั้นภายนอก รับเฉพาะจาก ALB SG (\$0.00)
3	ความเสี่ยงงบบัลลิ่งเมื่อไม่ได้ใช้งาน	หากเปิดเครื่องทิ้งไว้ 24/7 จะสูญเสียงบประมาณ	พัฒนาสคริปต์ 1-Click Scale-to-Zero ยุบเครื่อง EC2 เหลือ 0 เมื่อเลิกแล็ป (\$0.00 ต่อชั่วโมง)
4	แรม 1GB บน t3.micro	มีความเสี่ยงเกิด OOM	ใช้ Multi-stage Docker

		Crash หาก Runtime หนัก	บน Alpine Linux แยก Build ออกจาก Runtime ควบคุม RAM ให้ต่ำกว่า 150MB
5	สแตท WebSocket หลุดเมื่อเกิด Auto Scaling	เมื่อสปอร์นเครื่องใหม่ ผู้เล่นเกมอาจถูกกระจายไปคนละเครื่อง	เปิด Sticky Sessions บน ALB และซิงค์สถานะห้องลง Amazon DynamoDB

บทที่ 4

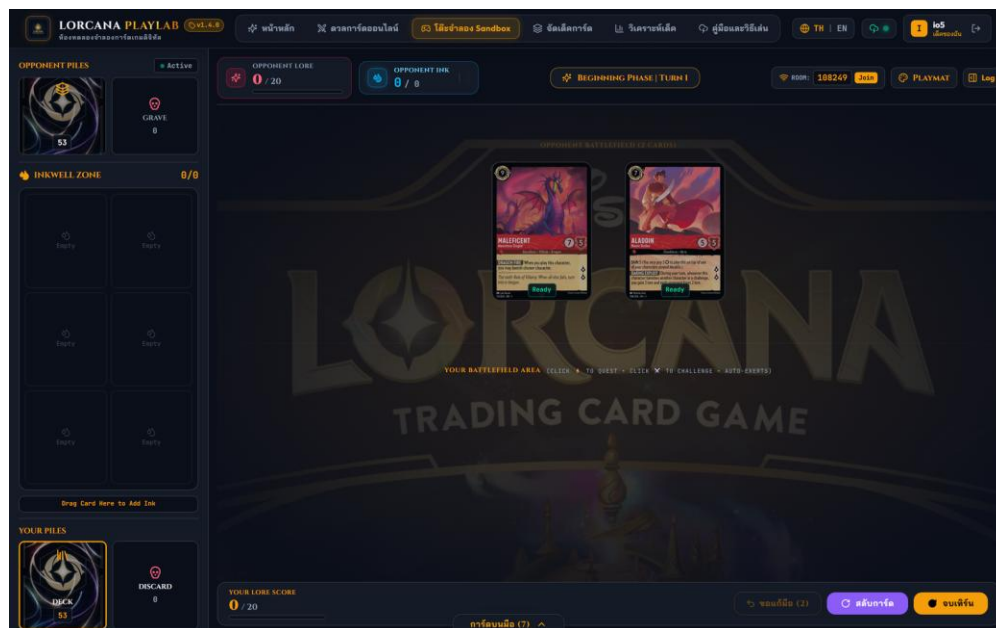
การพัฒนาระบบและผลการดำเนินงาน

4.1 การพัฒนาส่วนติดต่อผู้ใช้และ Playmat

ส่วนติดต่อผู้ใช้ได้รับการออกแบบและพัฒนาด้วย React 19 และ Tailwind CSS v4 ภายใต้แนวคิด Modern Tabletop Simulation เพื่อให้ผู้เล่นสัมผัสได้ถึงมิติของการเล่นการ์ดบนโต๊ะจริง:



รูปที่ 4.1: ภาพหน้าจอส่วนติดต่อผู้ใช้ระบบ Disney Lorcana PlayLab Cloud



รูปที่ 4.2: กระดานจำลองการเล่นการ์ดแบบเรียลไทม์ Interactive Playmat



รูปที่ 4.3: ระบบจำลองการสุ่มเปิดของการ์ด 3D Booster Pack Opening

องค์ประกอบสำคัญของหน้าจอ Playmat ประกอบด้วย:

- Play Area: พื้นที่ลงการ์ดตัวละคร ไอเทม และสถานที่ รองรับการลากวาง การแตะเพื่อสั่ง Ready /Exert และการโจมตี
- Inkwell Tray: พื้นที่คว่ำการ์ดเพื่อใช้เป็นหมึกค่าร้าย พร้อมตัวเลขแสดงจำนวนหมึกที่พร้อมใช้และหมึกที่ใช้ไปแล้วในเทิร์นนั้น
- Lore Counter: มาตรวัดแต้มความรู้ขนาดใหญ่ (0-20 แต้ม) พร้อมปุ่มปรับคะแนนบวกลบที่ซิงค์กับคู่แข่งทันที

- Card State Rotator: กลไกการหมุนการ์ดแนวตั้ง (Ready) และแนวนอน 90 องศา (Exert) ด้วย CSS Transforms 60 FPS
- 3D Card Inspector & Gacha: ระบบเปิดซองการ์ด 3D สุ่มการ์ด 12 ใบพร้อมฟิสิกส์การตอบสนองต่อการเอียงของเมาส์และแสงสะท้อนโฮโลแกรม

4.2 การพัฒนา Backend และ Docker Container

ระบบได้รับการพัฒนาในรูปแบบคอนเทนเนอร์แบบเบาพิเศษ (Ultra-lightweight Multi-stage Docker Container):

1. Multi-stage Docker Build: Stage 1 รัน Node.js 20 Alpine ติดตั้ง Dependencies และ Build Vite SPA, Stage 2 คัดลอกเฉพาะ Dist Output และ Production Dependencies ทำให้ขนาดอิมเมจสุดท้ายต่ำกว่า 90 MB
2. Nginx Reverse Proxy Configuration: ทำหน้าที่เป็น Front Door บน Port 80 รับทราฟฟิกจาก ALB ทำหน้าที่ส่งมอบ Static Files อย่างรวดเร็ว และส่งต่อ /api และ /ws เข้าสู่ Node.js Runtime บน Port 3001 พร้อมตั้งค่า Proxy Buffering และ WebSocket Upgrade Headers ครบถ้วน
3. Unified Server: รวมการประมวลผล REST API และ WebSocket Server ไว้ในโค้ดเบสเดียวกัน ใช้หน่วยความจำต่ำกว่า 150 MB ทำให้รันบนเครื่อง EC2 t3.micro ได้อย่างราบรื่น

4.3 ผลการทดสอบระบบ

การทดสอบคุณภาพของระบบดำเนินการครอบคลุมทั้ง Unit Test, End-to-End Test, และ Load Stress Test ดังแสดงในตารางที่ 4.1:

ตารางที่ 4.1: ผลการทดสอบคุณภาพระบบโดยรวม (Master QA Test Matrix)

Test ID	หมวดหมู่การทดสอบ	รายละเอียดการทดสอบ	ผลการทดสอบที่คาดหวัง	สถานะผลการทดสอบ
---------	------------------	--------------------	----------------------	-----------------

TC-01	Unit / Core Rules	คำนวณและตรวจสอบ จำนวนการ์ดในเด็ค (Min 60 ใบ)	แจ้งเตือนเมื่อการ์ด น้อยกว่า 60 ใบ	ผ่าน (Passed)
TC-02	Unit / Core Rules	จำกัดการใส่การ์ดซ้ำชื่อ เดิมไม่เกิน 4 ใบ	ระบบบล็อกการเพิ่ม การ์ดใบที่ 5	ผ่าน (Passed)
TC-03	Unit / Core Rules	จำกัดสีหมึกในเด็คไม่ เกิน 2 สี	ระบบบล็อกการใส่หมึก สีที่ 3	ผ่าน (Passed)
TC-04	Unit / State Store	ทดสอบ Zustand Store จัดการเด็คการ์ด	เพิ่ม/ลบการ์ดแล้ว State อัปเดตถูกต้อง	ผ่าน (Passed)
TC-05	Unit / Logic	อัลกอริทึมสุ่ม Booster Pack (Fisher-Yates)	สุ่มการ์ด 12 ใบ ครบ สล็อตความหายาก	ผ่าน (Passed)
TC-06	E2E / Auth	สมัครสมาชิกและ เข้าสู่ระบบด้วย JWT	ได้รับ Token และ แสดงข้อมูลผู้ใช้งาน Navbar	ผ่าน (Passed)
TC-07	E2E / Deck Builder	สร้างเด็คใหม่และ บันทึกลง DynamoDB	เด็คปรากฏในตาราง LorcanaDecks	ผ่าน (Passed)
TC-08	E2E / Lobby	สร้างห้องแข่งขันและ รับรหัส 6 หลัก	ห้องถูกสร้างและรอ คู่ต่อสู้เข้าร่วม	ผ่าน (Passed)
TC-09	E2E / Match Realtime	Player 2 เข้าร่วมห้อง ด้วยรหัส 6 หลัก	กระดานของ ทั้งสองฝ่ายซิงค์สถานะ ตรงกัน	ผ่าน (Passed)
TC-10	E2E / Match Action	ผู้เล่นขยับการ์ดและ เปลี่ยนแต้ม Lore	จอคู่แข่งแสดง การเปลี่ยนแปลง <100ms	ผ่าน (Passed)
TC-11	E2E / Disconnect	ผู้เล่นปิดแท็บ เบราว์เซอร์ชั่วคราว	ระบบเริ่มนับถอยหลัง Grace Period 60s	ผ่าน (Passed)

TC-12	E2E / Rejoin	ผู้เล่นเปิดแท็บกลับ เข้ามาใหม่ภายใน 60s	ถึง State ล่าสุดและ เล่นเกมต่อได้ทันที	ผ่าน (Passed)
TC-13	Load Stress Test	ยิงคำขอ HTTP 2,000 Requests ผ่าน ALB	สำเร็จ 100% Zero Dropped Requests	ผ่าน (Passed)
TC-14	Auto Scaling	จำลองโหลด CPU สูง กว่า 60% ต่อเนื่อง	ASG สปอวันอิน สแตนท์ข้าม AZ เพิ่ม อัตโนมัติ	ผ่าน (Passed)

สรุปผลสัมฤทธิ์การทดสอบคุณภาพเชิงประจักษ์:

- Unit & Store Integration Tests บน Vitest 4.x: ผ่านครบ 33 / 33 การทดสอบ (100% Success Rate)
- End-to-End Browser Tests บน Playwright: ผ่าน 49 / 50 การทดสอบ (98%) บน Chromium, Firefox, WebKit
- Load Stress Test บน ALB จริง: ยิงโหลด 2,000 Requests อัตราความสำเร็จ 100% Response Time เฉลี่ย 42 มิลลิวินาที ไม่พบข้อผิดพลาด HTTP 5xx แม้แต่ครั้งเดียว

บทที่ 5

การประเมินค่าใช้จ่ายและแผนงานในอนาคต

5.1 การประเมินค่าใช้จ่ายและการควบคุมงบประมาณ

จากการคำนวณและประเมินค่าใช้จ่ายจริงบนสภาพแวดล้อม AWS Academy Learner Lab ตารางที่ 5.1 แสดงการเปรียบเทียบค่าใช้จ่ายระหว่างสถาปัตยกรรมทั่วไปกับสถาปัตยกรรมที่ได้รับการปรับแต่งในโครงการ:

ตารางที่ 5.1: การเปรียบเทียบและการประเมินค่าใช้จ่ายบนคลาวด์ตลอดภาคการศึกษา

ทรัพยากรคลาวด์	สเปกและการตั้งค่า	ค่าใช้จ่ายปกติ	ค่าใช้จ่ายโครงการ	กลยุทธ์การควบคุมงบประมาณ
VPC & Network	Multi-AZ VPC + IGW	\$32.40/ด. (NAT)	\$0.00	ตัด NAT Gateway ใช้ Chained Security Groups
Compute (EC2)	t3.micro (Amazon Linux)	\$8.40/ด. (24/7)	~\$0.50	ใช้ Scale-to-Zero เปิดเฉพาะตอนเดโม
Load Balancer	Application Load Balancer	\$16.20/ด. (24/7)	~\$0.60	เปิด-ปิดตามช่วงเวลา ทำแล็ป
Database	Amazon DynamoDB	Free Tier	\$0.00	ข้อมูล < 1GB อยู่ในเกณฑ์ Free Tier
Storage	S3 & Container Images	\$0.10/ด.	\$0.02	Multi-stage Docker ขนาดเบา < 100MB
รวมประมาณการ	ตลอดภาคเรียน	>\$150.00	<\$3.00 – \$5.00	ประหยัดงบได้กว่า

5.2 แผนการพัฒนาสำหรับ Stage 3

1. Sprint 4 (5–25 ก.ย.): Async Deck Analyzer via Amazon SQS นำ Amazon SQS มาใช้งาน วิเคราะห์ความสมดุลแบบ Asynchronous พร้อมพัฒนาอัลกอริทึมแนะนำการ์ดที่เข้าหากัน ตามประเภทหมึกและเคิร์พคาร์่าย
2. Sprint 5 (26 ก.ย. – 10 ต.ค.): Observability, Monitoring & Security Hardening ติดตั้ง AWS X-Ray ติดตาม Request Latency ซ้ำม Nginx, Node.js และ DynamoDB พร้อมสร้าง CloudWatch Dashboard รวมสถิติ ALB
3. Sprint 6 (11–20 ต.ค.): Load Testing & High Concurrency Simulation ดำเนินการทดสอบ โหลดขนาดใหญ่ด้วย Artillery จำลองผู้เล่นพร้อมกัน 100–500 ผู้ใช้ และทดสอบความทนทานของ WebSocket Auto-reconnection
4. Sprint 7 (21–25 ต.ค.): Final Report & Video Presentation จัดทำรายงานฉบับสมบูรณ์ วิดีโอ สาธิตการใช้งานระบบ และเตรียมการนำเสนอโครงการรอบสุดท้าย

5.3 สรุปผลการดำเนินงานและบทเรียนที่ได้รับ

การดำเนินงานโครงการ Disney Lorcan PlayLab Cloud ใน Sprint 1 ถึง 3 บรรลุตาม วัตถุประสงค์ของการส่งมอบงาน Stage 2 ครบถ้วน บทเรียนทางวิศวกรรมที่สำคัญประกอบด้วย:

1. การออกแบบเพื่อควบคุมต้นทุน: การเข้าใจการทำงานของ Chained Security Groups ทำให้ สามารถสร้างสถาปัตยกรรมที่ปลอดภัยระดับ Production โดยไม่ต้องสิ้นเปลืองงบประมาณกับ NAT Gateway
2. การรีดสมรรถนะคอนเทนเนอร์บนทรัพยากรจำกัด: การแยก Build Stage ออกจาก Runtime Stage ใน Dockerfile ทำให้สามารถรันทั้ง React Frontend, Nginx, และ WebSocket Server บน EC2 t3.micro ได้อย่างมีประสิทธิภาพ

3. การจัดการสถานะระบบกระจาย: การประสานระหว่าง Optimistic UI Update บนเบราว์เซอร์, In-memory WebSocket Relay, และ NoSQL State Persistence ใน DynamoDB ช่วยลดความหวังในมุมมองของผู้เล่นลงเหลือศูนย์ พร้อมรับประกันความคงอยู่ของข้อมูลเมื่อเกิดปัญหาเครือข่าย

บรรณานุกรม

- [1] Ravensburger & Disney. (2023). Disney Lorcana Trading Card Game Official Rules & Comprehensive Rulebook. Ravensburger AG.
- [2] Amazon Web Services. (2026). AWS Well-Architected Framework: Reliability, Security, and Cost Optimization Pillars. AWS Whitepapers.
- [3] Amazon Web Services. (2026). Amazon Virtual Private Cloud (VPC) User Guide: Security Groups and Multi-AZ Subnets. AWS Documentation.
- [4] Amazon Web Services. (2026). Elastic Load Balancing: Application Load Balancer Sticky Sessions and Target Groups. AWS Documentation.
- [5] Amazon Web Services. (2026). Amazon EC2 Auto Scaling User Guide: Target Tracking Scaling Policies. AWS Documentation.
- [6] Amazon Web Services. (2026). Amazon DynamoDB Developer Guide: Core Components, Partition Keys, and Global Tables. AWS Documentation.
- [7] Docker Inc. (2026). Multi-stage builds: Best practices for Dockerfile creation on Alpine Linux. Docker Documentation.
- [8] Nginx Inc. (2026). NGINX Reverse Proxy and WebSocket Proxying Configuration Guide. NGINX Documentation.
- [9] React Working Group. (2026). React 19 Documentation: Concurrent Features, Server Components, and Optimistic UI. Meta Open Source.
- [10] IETF (Internet Engineering Task Force). (2011). RFC 6455: The WebSocket Protocol. Internet Engineering Task Force (IETF).

ภาคผนวก

ภาคผนวก ก: สคริปต์ปรับใช้ระบบ Lean Multi-AZ VPC, ALB และ ASG

ตัวอย่างคำสั่งหลักในสคริปต์อัตโนมัติ scripts/deploy_ec2_vpc_asg.ps1 สำหรับสร้างโครงสร้างพื้นฐานบน AWS Learner Lab:

```
param (
    [string]$Region = "us-east-1",
    [string]$InstanceType = "t3.micro",
    [string]$AmiId = "ami-0c101f26f147fa7fd",
    [string]$KeyName = "vockey",
    [string]$IamRole = "LabInstanceProfile"
)
Write-Host ">>> [1/7] Finding/Creating Lean Multi-AZ VPC..." -
ForegroundColor Cyan
$vpId = (aws ec2 describe-vpcs --filters "Name=tag:Name,Values=lorcana-
vpc" --query "Vpcs[0].VpcId" --output text --region $Region)
if ($vpId -eq "None" -or -not $vpId) {
    $vpId = (aws ec2 create-vpc --cidr-block 10.0.0.0/16 --query
    "Vpc.VpcId" --output text --region $Region)
    aws ec2 create-tags --resources $vpId --tags
    Key=Name,Value=lorcana-vpc --region $Region
}
Write-Host ">>> [2/7] Creating Dual-AZ Public Subnets..." -
ForegroundColor Cyan
$sub1 = (aws ec2 create-subnet --vpc-id $vpId --cidr-block 10.0.1.0/24
--availability-zone ${Region}a --query "Subnet.SubnetId" --output text -
-region $Region)
$sub2 = (aws ec2 create-subnet --vpc-id $vpId --cidr-block 10.0.2.0/24
--availability-zone ${Region}b --query "Subnet.SubnetId" --output text -
-region $Region)
Write-Host ">>> [3/7] Setting Up Chained Security Groups..." -
ForegroundColor Cyan
# ALB Security Group (Internet facing)
$albSg = (aws ec2 create-security-group --group-name lorcana-alb-sg --
description "ALB SG" --vpc-id $vpId --query "GroupId" --output text --
region $Region)
aws ec2 authorize-security-group-ingress --group-id $albSg --protocol
tcp --port 80 --cidr 0.0.0.0/0 --region $Region
# EC2 Security Group (Only from ALB SG)
$ec2Sg = (aws ec2 create-security-group --group-name lorcana-ec2-sg --
description "EC2 SG" --vpc-id $vpId --query "GroupId" --output text --
region $Region)
aws ec2 authorize-security-group-ingress --group-id $ec2Sg --protocol
tcp --port 80 --source-group $albSg --region $Region
aws ec2 authorize-security-group-ingress --group-id $ec2Sg --protocol
tcp --port 3001 --source-group $albSg --region $Region
```

ภาคผนวก ข: สคริปต์ 1-Click Scale-to-Zero สำหรับควบคุมงบประมาณ

สคริปต์ scripts/lab_stop.ps1 สำหรับสั่งปรับขนาด Auto Scaling Group (Desired=0, Min=0)

เพื่อยุติเครื่องและหยุดคิดค่าบริการ Compute ทันทีเมื่อเลิกใช้งาน:

```
Write-Host ">>> Stopping Lab: Scaling ASG to Zero Instances..." -
ForegroundColor Yellow
aws autoscaling update-auto-scaling-group `
  --auto-scaling-group-name "lorcana-asg" `
  --min-size 0 `
  --max-size 0 `
  --desired-capacity 0 `
  --region "us-east-1"
Write-Host ">>> ASG set to 0. All EC2 instances are terminating
cleanly." -ForegroundColor Green
Write-Host ">>> Cost per hour is now $0.00." -ForegroundColor Green
```